

Fair Drop — Data Model

There are two stores. They have **different roles** (see):

- **DropDO SQLite** (per drop): authoritative live state. Small, minimal indexes (index writes count against quota).
- **D1** (global): system of record across drops + final projections of each drop + experiments + audit.

Conventions: IDs are prefixed ULIDs (`usr_`, `ses_`, `drp_`, `par_`, `tkr_`, `res_`, `exp_`, `aud_`).

Timestamps are INTEGER epoch ms. In virtual-clock drops, drop-domain timestamps are virtual ms and are flagged by `drops.clock_mode`. JSON columns are TEXT validated by zod in `packages/shared`.

Part A — D1 (global)

A1. `events`

- **Purpose:** the public event (FUTUREFEST 2026).
- **PK** `id` (slug, e.g. `futurefest-2026`).
- Columns: `name`, `venue`, `starts_at`, `description`, `image_url`, `created_at`.
- **Invariants:** immutable after the first drop is published.

A2. `drops`

- **Purpose:** one sale instance of an event (the consumer demo drop or an experiment run).
- **PK** `id`. **FK** `event_id` → `events.id`, `experiment_id` → `experiments.id` NULL.
- Columns: `mode` (FAIR|NAIVE), `kind` (PUBLIC|EXPERIMENT), `clock_mode` (WALL|VIRTUAL), `state`, `total_inventory`, `opens_at`, `registration_closes_at`, `booking_closes_at`, `offer_ttl_ms`, `hold_ttl_ms`, `config_json` (rate limits, risk thresholds, capacity model), `seed_commitment`, `allocation_algorithm`, `allocation_committed_at`, `created_at`, `updated_at`.
- **Indexes:** (`event_id`, `state`), (`experiment_id`).
- **Status field:** `state` (projection of the DO drop state).
- **Invariants:** `config_json`, `total_inventory` and `seed_commitment` are immutable once `state` ≥ SCHEDULED. `kind=EXPERIMENT` ⇔ `experiment_id` NOT NULL.

A3. users

- **Purpose:** logical account (consumer). Synthetic accounts for sim drops are **not** stored here (they live in DO chunk storage). See OD-2.
- **PK** `id`. Columns: `display_name`, `email_hash`, `email_display` NULL, `created_at`, `status` (ACTIVE|SUSPENDED).
- **Unique:** `email_hash`.
- **Invariants:** one account per normalized email.

A4. sessions

- **Purpose:** session recovery, revocation, concurrent-session counting.
- **PK** `id`. **FK** `user_id` → `users.id`.
- Columns: `device_id`, `network_key`, `user_agent_hash`, `created_at`, `last_seen_at`, `expires_at`, `status` (ACTIVE|REVOKED|EXPIRED), `revoke_reason`.
- **Indexes:** (`user_id`, `status`).
- **Invariants:** the cookie HMAC must match `id` + `user_id`. A revoked session is never reactivated. `last_seen_at` is updated at most once per 5 min (write budget).

A5. participants (projection)

- **Purpose:** final/periodic projection of drop participants, for consumer drops and for auditing. Logical-sim participants are projected as chunks into `experiment_artifacts`, not here.
- **PK** `id`. **FK** `drop_id` → `drops.id`, `user_id` → `users.id`.
- Columns: `status`, `registered_at`, `eligibility_reason`, `risk_action`, `risk_score`, `rank_key`, `waitlist_rank`, `offer_expires_at`, `updated_at`.
- **Unique:** (`drop_id`, `user_id`) ← I-3 at rest.
- **Indexes:** (`drop_id`, `status`).

A6. tickets (projection)

- **Purpose:** final/periodic state of each ticket.
- **PK** `id`. **FK** `drop_id`, `holder_participant_id` NULL → `participants.id`.
- Columns: `seq` (1..N), `status` (AVAILABLE|HELD|CONFIRMED), `updated_at`.
- **Unique:** (`drop_id`, `seq`). Partial unique (`drop_id`, `holder_participant_id`) WHERE `holder_participant_id` IS NOT NULL ← I-4.
- **Invariant:** per drop, `COUNT(*)` = `total_inventory`.

A7. allocations

- **Purpose:** immutable lottery output (winners + initial waitlist head). Promotions are recorded as additional rows.
- **PK** (drop_id, participant_id).
- Columns: outcome (WINNER|WAITLIST|PROMOTED), rank (0-based position in order), rank_key, created_at.
- **Unique:** (drop_id, rank).
- **Invariants:** append-only. COUNT(outcome=WINNER) = min(N, eligible). For 50k logical runs only WINNER + PROMOTED rows are stored. The full order is recomputable from the snapshot + seed.

A8. reservations (projection)

- **PK** id . **FK** drop_id, participant_id, ticket_id.
- Columns: status (HELD|CONFIRMED|EXPIRED|RELEASED), idempotency_key, created_at, expires_at, confirmed_at, ended_at.
- **Unique (partial):** (drop_id, participant_id) WHERE status IN ('HELD', 'CONFIRMED').
- **Indexes:** (drop_id, status).

A9. risk_decisions

- **Purpose:** explainable record of risk **action changes** (not every score update).
- **PK** id . **FK** drop_id, participant_id NULL, session_id NULL.
- Columns: subject_ref (participant/session/sim account ref), action, score, reasons_json, engine_version, created_at.
- **Indexes:** (drop_id, action).
- **Invariant:** reasons_json must be non-empty for any action ≠ ALLOW. For logical runs only the first N=2,000 changes are stored (rest aggregated in metrics); the cap is recorded in the experiment.

A10. experiments

- **Purpose:** one simulated run (one mode). Paired runs share comparison_id.
- **PK** id . **FK** drop_id → drops.id.
- Columns: comparison_id, mode (FAIR|NAIVE), sim_mode (LOGICAL|REAL_HTTP), scenario, status, config_json (full scenario + profile mix + amplification + capacity model + risk config + timings), population_seed, population_hash, labels_hash, simulator_version,

server_version (git sha), risk_engine_version, started_at, finished_at, results_json, created_by.

- **Indexes:** (comparison_id), (status, created_at).
- **Invariants:** config_json, population_seed and population_hash are immutable after CREATED. results_json is written only by the server at COMPLETED.

A11. experiment_metrics

- **Purpose:** time-series snapshots (dashboard replay) + final scalar metrics.
- **PK** (experiment_id, t_ms, metric_set). metric_set ∈ {snapshot, final, client_latency}.
- Columns: values_json (counters, rates, histograms).
- **Write budget:** snapshot every 2 s (wall) or every 10 batches (logical), so a run produces ≤ ~150 rows.

A12. experiment_artifacts

- **Purpose:** chunked large blobs needed for reproduction: eligible snapshot list, sim participant states, label map, final ticket table.
- **PK** (experiment_id, kind, chunk_index). Columns: content_type, sha256, data (TEXT/BLOB ≤ 1 MB).
- **Invariant:** SHA-256 over the concatenated chunks of kind=snapshot equals audit_records.snapshot_hash.

A13. audit_records

- **Purpose:** everything a judge needs to verify an allocation.
- **PK** id. **FK** drop_id (unique), experiment_id NULL.
- Columns: allocation_algorithm, algorithm_version, seed_commitment, seed_committed_at, server_seed (NULL until revealed), revealed_at, snapshot_hash, eligible_count, total_inventory, final_seed, result_hash, winners_hash, allocation_committed_at, integrity_json (counters + scan results: oversells, duplicate allocations, drift, invalid reservations), code_version, created_at.
- **Invariants:** SHA-256(server_seed) = seed_commitment. Row is append-only after revealed_at (integrity_json is updated once at CLOSE).

Part B — DropDO SQLite (per drop, authoritative)

Table	Purpose	PK	Unique / indexes	Notes
meta	Single row: drop config copy, state, clock, seed (secret), commitment, snapshot_hash, counters checkpoint	k	—	state changes only through lifecycle
participants	Consumer/real-HTTP participants	id	UNIQUE participant_key . Index status	One row per participant
participant chunks	Logical-sim participants, packed ($\leq 1,000$ /row)	chunk_index	—	Rewritten only when a chunk changes (registration batch, freeze, allocation, booking batch)
tickets	500 rows	seq	partial UNIQUE holder_participant_id	Only inventory/writes
reservations	Holds/confirmations	id	partial UNIQUE participant id WHERE status IN (HELD,CONFIRMED) . Index (status, expires_at)	Drives expiry
idempotency	Persisted keys for reserve/confirm	(participant id , key)	—	Body hash + response
outbox	Pending D1 projection writes	autoinc seq	—	Flushed in batches. Deleted after ack
risk state	Last action per subject (only when $\neq$ ALLOW)	subject_ref	—	Features themselves stay in memory

In-memory only (rebuildable): token buckets, live feature windows, metric counters, latency histograms, event-feed ring buffer (last 500), participant map for logical drops (hydrated from chunks on wake).

Write-budget estimate (50k logical, FAIR)

Item	DO rows	D1 rows
Registration chunks (50 × ~3 rewrites)	~150	—
Tickets init + transitions	~500 + ~1,500	500 (final)
Reservations	~1,000	~600
Allocations	—	~600
Snapshot/artifact chunks	—	~100
Metrics snapshots	—	~150
Risk decisions (capped)	~2,000	≤ 2,000
Total	~5k	~4k

That allows roughly 10+ full paired runs per day on the free tier. The real numbers are measured and reported per run as `rows_written_do` / `rows_written_d1`.

Part C — Key invariants restated as checks

```
sql
-- I-5 (DO)
SELECT SUM(status='AVAILABLE')+SUM(status='HELD')+SUM(status='CONFIRMED') = :total FROM
tickets;
-- I-4 duplicate holder (must return 0 rows)
SELECT holder_participant_id FROM tickets WHERE holder_participant_id IS NOT NULL GROUP BY
1 HAVING COUNT(*)>1;
-- Oversell (must be 0)
SELECT MAX(0, (SELECT COUNT(*) FROM reservations WHERE status IN ('HELD','CONFIRMED')) -
:total);
-- Invalid reservation (must return 0 rows): reservation for a participant never ALLOCATED,
or confirmed after expiry
SELECT r.id FROM reservations r JOIN participants p ON p.id=r.participant_id
WHERE p.status NOT IN ('HELD','CONFIRMED','HOLD_EXPIRED','RELEASED') OR (r.confirmed_at >
r.expires_at);
```